Analysis of One Privacy-Preserving Electricity Data Classification Scheme Based on CNN Model With Fully Homomorphism

Zhengjun Cao, Lihua Liu

Abstract. We show that the data classification scheme [IEEE Trans. Sustain. Comput., 2023, 8(4), 652-669)] failed to check the compatibility of encoding algorithm and homomorphic encryption algorithm. Some calculations should be revised to ensure all operands are first encoded using the same scaling factors. The canonical embedding map depending on the natural projection should be explicitly arranged so as to construct an efficient decoding algorithm.
Keywords: Homomorphic encryption, data classification, scaling factors, canonical embedding map, natural projection.

1 Introduction

Recently, Xia et al. [1] have presented a scheme for fog computing-based smart metering system, which can realize the classification of encrypted electricity consumption data by fog nodes so as to reduce the classification costs on the cloud server. In the scheme, the cloud server is used to train the convolutional neural network (CNN) classification model, and the fog nodes are used to perform the classification of ciphertext-based electricity consumption data generated by fully homomorphic method. Though the scheme is interesting, we find it is flawed due to some inconsistent computations. We also provide some suggestions for its revision.

2 The Cheon et al.'s homomorphic encryption

In 2017, Cheon et al. [2] have presented a homomorphic encryption (CKKS) for arithmetic of approximate numbers, which will be used for the Xia et al. scheme. Let $\lfloor r \rceil$ be the nearest integer to the real number r. For an integer q, let $\mathbb{Z}_q$ be the set $\mathbb{Z} \cap (-q/2, q/2]$, $[z]_q = (z \bmod q)$. For a positive integer M, let $\Phi_M(X)$ be the M-th cyclotomic polynomial of degree $N = \phi(M)$ where $\phi(\cdot)$ is the Euler function. Let $\mathbb{Z}_M^* = \{x \in \mathbb{Z}_M : \gcd(x, M) = 1\}$,

$$\mathcal{R} = \mathbb{Z}[X]/(\Phi_M(X)),\ \mathcal{R}_q = \mathbb{Z}[X]/(\Phi_M(X), q),$$

and $\mathcal{S} = \mathbb{R}[X]/(\Phi_M(X))$ be the set of $a(X) = \sum_{j=0}^{N-1} a_j X^j$, where $(a_0, \cdots, a_{N-1}) \in \mathbb{R}^N$.

Z. Cao, Department of Mathematics, Shanghai University, Shanghai, 200444, China.
L. Liu, Department of Mathematics, Shanghai Maritime University, Shanghai, 201306, China.
Email: liulh@shmtu.edu.cn

The *canonical embedding map* σ is defined by

$$\sigma : \quad a \in \mathcal{S} \mapsto (a(\zeta_M^{j_1}), a(\zeta_M^{j_2}), \cdots, a(\zeta_M^{j_N})) \in \mathbb{C}^N, j_i \in \mathbb{Z}_M^*$$

where $\zeta_M = \exp(-2\pi i/M)$, i.e., a primitive M-th roots of unity. Let T be a multiplicative subgroup of $\mathbb{Z}_M^*$ such that $\mathbb{Z}_M^*/T = \{1, -1\}$. The *natural projection* π is defined by

$$\pi : \ (z_j)_{j \in \mathbb{Z}_M^*} \in \mathcal{R} \mapsto (z_j)_{j \in T} \in \mathbb{C}^{N/2}$$

Hence, $m(X) \in \mathcal{R}$ can be transformed into a complex vector $\pi \circ \sigma(m) \in \mathbb{C}^{N/2}$. Define $\pi^{-1}(\boldsymbol{z}) = [\boldsymbol{z}, \bar{\boldsymbol{z}}]^T$. Use some round-off algorithm to discritize $\pi^{-1}(\boldsymbol{z})$ to an element of $\sigma(\mathcal{R})$, and denote it by $\lfloor \pi^{-1}(\boldsymbol{z}) \rceil_{\sigma(\mathcal{R})}$. The encoding/decoding algorithms are explicitly described as follows.

- **Ecd**$(\boldsymbol{z}; \triangle)$. For the scaling factor $\triangle$ and a $(N/2)$-dimensional vector $\boldsymbol{z} = (z_i)_{i \in T}$ of complex numbers, output $m(X) = \lfloor \sigma^{-1}(\triangle \cdot \pi^{-1}(\boldsymbol{z})) \rceil_{\sigma(\mathcal{R})} \in \mathcal{R}$.
- **Dcd**$(m; \triangle)$. For a polynomial $m \in \mathcal{R}$, output the vector $\boldsymbol{z} = \pi \circ \sigma(\triangle^{-1} \cdot m)$.

The following homomorphic encryption (HE) scheme is defined over the integer polynomial ring $\mathcal{R}$. We refer to the original descriptions [2] for the phases of **KeyGen**, $\mathbf{Enc}_{pk}(m)$, $\mathbf{Dec}_{sk}(\boldsymbol{c})$, $\mathbf{Add}(\boldsymbol{c}_1, \boldsymbol{c}_2)$, $\mathbf{Mult}_{evk}(c_1, c_2)$.

3 Review of the Xia et al.'s scheme

In the considered scenario, there are four entities: certification authority (CA), cloud server (CS), fog node (FN) and smart meters (SMs). Let N be a power-of-two integer, q be a prime number of ciphertext coefficient modulus, L be the calculation depth w.r.t. the integer p, Q_l be $\prod_{i=1}^{l} p^i, l \in \{1, 2, \cdots, L\}$. Let pka_i, ska_i be public key, secret key of the i-th smart meter, pkr_i, skr_i be public key, secret key of the i-th manager region, evk_i be the evaluation key of the i-th manager region. $e_{i,j}$ is data of the j-th acquisition point of the i-th day, and n, m are the row number, column number of data. Let P_j, CT_j be electricity consumption plaintext matrix, ciphertext matrix of j-th smart meter.

Key distribution. The CA generates the homomorphic encryption public/private key pair (pkr_r, skr_r) and evk_r for each management region r, where (pkr_r, skr_r) and evk_r are securely sent to the cloud server, and the corresponding fog node fnr, respectively. CA generates the public/private key pair (pka_g, ska_g) for the registered smart meter sm_g.

Data Encryption and Signing. The smart meter sm_g encodes its electricity consumption data

$$E = \begin{bmatrix} e_{1,1} & \cdots & e_{1,m} \\ \vdots & \ddots & \vdots \\ e_{n,1} & \cdots & e_{n,m} \end{bmatrix}$$

Concretely, $e_1 = (e_{1,1}, e_{2,1}, \cdots, e_{n,1})$ is encoded as

$$\begin{aligned} Encode(e_1) &= \triangle * [e_{1,1}, e_{2,1}, \cdots, e_{n,1}, 0, 0, \cdots, 0] * M_\tau^{-1} \\ &= (p_0, p_1, \cdots, p_{\tau-1}), \end{aligned}$$

$$M_\tau = \begin{bmatrix} \varepsilon^0 & \varepsilon^1 & \ddots & \varepsilon^{\tau-1} \\ \varepsilon^{0*5} & \varepsilon^{1*5} & \ddots & \varepsilon^{(\tau-1)*5} \\ \vdots & \vdots & \vdots & \vdots \\ \varepsilon^{0*5} & \varepsilon^{1*(4\tau-3)} & \ddots & \varepsilon^{(\tau-1)*(4\tau-3)} \end{bmatrix} \tag{1}$$

The matrix E is encoded as $P_g = (pt_1, pt_2, \cdots, pt_m)$. Use the region manager r's public key pkr_r to encrypt P_g as $CT_g = En_{pkr_r}(P_g)$. Use its private key ska_g to sign the matrix CT_g and generate the signature $sg_g = Sign_{ska_g}(CT_g)$. Send $CT_g \| sg_g$ to a target fog node fn_r.

Data Encryption and Signing. The fog node fn_r aggregates the signatures in the region r to obtain $saggre_r$. Then compute $CT_{CK} = En_{pkr_r}(CK)$, where $CK = (d_1, d_2, \cdots, d_n)$ is the model parameter, and compute

$$\begin{aligned} ct_{one} &= CKKS.Mult(CT_g, CT_{CK}) \\ &= \left(En \begin{bmatrix} e_{1,1} \\ e_{2,1} \\ \vdots \\ e_{n,1} \end{bmatrix}, \cdots, En \begin{bmatrix} e_{1,m} \\ e_{2,m} \\ \vdots \\ e_{n,m} \end{bmatrix} \right) * En \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_n \end{bmatrix} \\ ct_{two} &= CKKS.Rescale(ct_{one}) \\ ct_{three} &= [ct_{three}(1), \cdots, ct_{three}(m)] \\ &\cdots \\ ct_{fourteen} &= CKKS.Rescale(ct_{thirteen}) \end{aligned}$$

We refer to the original description (pp. 660-663, Ref.[1]).

Extraction of Classification Results. See page 663, Ref.[1].

4 Analysis of the Xia et al.'s scheme

4.1 An example for CKKS encoding mechanism

Let $M = 8, \triangle = 64, T = \{\zeta_8, \zeta_8^3\}$, where $\zeta_8 = \exp(-2\pi i/8)$. For a vector $\boldsymbol{z} = (3 + 4i, 2 - i)$, expand it to $\pi^{-1}(\boldsymbol{z}) = [\boldsymbol{z}, \bar{\boldsymbol{z}}]^T = [3 + 4i, 2 - i, 3 - 4i, 2 + i]^T$. Then

$$\sigma = \begin{bmatrix} 1 & \zeta_8 & \zeta_8^2 & \zeta_8^3 \\ 1 & \zeta_8^3 & \zeta_8^6 & \zeta_8^9 \\ 1 & \zeta_8^{-1} & \zeta_8^{-2} & \zeta_8^{-3} \\ 1 & \zeta_8^{-3} & \zeta_8^{-6} & \zeta_8^{-9} \end{bmatrix} \tag{2}$$

$\sigma^{-1} * \triangle * \pi^{-1}(\boldsymbol{z}) = [160, -32\sqrt{2}, -160, -64\sqrt{2}]^T$. Round it to get $[160, -45, -160, -91]^T$. Output the polynomial $m(X) = 160 - 45X - 160X^2 - 91X^3$. To decode $m(X)$, compute $m(\zeta_8)/\triangle \approx 3.00823 + 4.0026i$, $m(\zeta_8^3)/\triangle \approx 1.99177 - 0.997398i$, round them to obtain $\boldsymbol{z}$.

The encoding algorithm is proved to be approximately homomorphic [2], compatible with the CKKS homomorphic encryption algorithm. But we find the encoding mechanism is unfamiliar to some newcomers.

4.2 Some typos

As we see, the canonical embedding map σ plays a key role in the CKKS encoding mechanism. In the Xia et al.'s scheme, the map is specified as the matrix M_τ. It also specifies that

> $\varepsilon = \exp(\pi/\tau)$, M_τ *is the matrix with* $\frac{\tau}{2}$ *rows and* τ *columns,* M_τ^{-1} *is the inverse of the matrix* M_τ*. Additionally, during the calculation process, the electricity consumption data vector* e_1 *is filled with* $\frac{\tau}{2} - n$ *zeros* (see page 659, Ref.[1]).

Clearly, the specification is wrong. It should be revised as

> *where* $\tau = N, \varepsilon = \exp(-\pi i/\tau)$, M_τ *is the matrix with* τ *rows and* τ *columns, and the vector* e_1 *is filled with* $\tau - n$ *zeros.*

4.3 The flawed calculations

The representation of embedding map σ, an $N \times N$ matrix, depends on the choice of expanding a target $N/2$-dimensional vector to an N-dimensional vector. For example, $\boldsymbol{z} = (3 + 4i, 2 - i)$ can be expanded as either

$$[3 + 4i, 2 - i, 3 - 4i, 2 + i]^T$$

or

$$[3 + 4i, 3 - 4i, 2 - i, 2 + i]^T$$

so as to construct an efficient decoding algorithm. In this case, each row of matrix M_τ should be explicitly arranged. But the proposed matrix Eq.(1) is simply arranged. It could be revised such that the second column is

$$[\varepsilon, \varepsilon^3, \cdots, \varepsilon^{N-1}, \varepsilon^{-1}, \varepsilon^{-3}, \cdots, \varepsilon^{-(N-1)}]^T$$

The raw data e_1 is first transformed into $Encode(e_1)$, which is then encrypted as $En(Encode(e_1))$. So, the model parameter $(d_1, d_2, \cdots, d_n)$ should also be first encoded as

$$(Encode(d_1), Encode(d_2), \cdots, Encode(d_n))$$

It is then encrypted as $En(Encode(d_1), Encode(d_2), \cdots, Encode(d_n))$. Otherwise, the cloud server will fail to retrieve the right classification results after the decoding procedure in the extraction phase. Therefore, the calculations for

$$\begin{aligned}
&ct_{one} = CKKS.Mult(CT_g, CT_{CK}),\\
&ct_{four} = CKKS.Mult_Plain(ct_{three}, c_1),\\
&ct_{seven} = CKKS.Mult_Plain(ct_{six}, c_2),\\
&ct_{nine}(i) = CKKS.Mult(ct_{eight}(i), ct_{W^1}(i)),\\
&ct_{thirteen} = CKKS.Mult(ct_{twelve}, ct_{w^2}(i)),
\end{aligned}$$

should be revised to ensure that all operands are already encoded by the same encoding algorithm, including the same scaling factors and rescaling factors. All these factors should be sent to the cloud server for the final extraction.

5 Conclusion

We show that there are some flaws in the Xia et al.'s data classification scheme, due to the misunderstanding about the Cheon et al.'s homomorphic encryption for arithmetic of approximate numbers. We also suggest a revising method. The findings in this note could be helpful for the future work on designing privacy-preserving data classification schemes.

References

[1] Z. Xia, D. Yin, K. Gu, X. Li, "Privacy-preserving electricity data classification scheme based on CNN model with fully homomorphism", *IEEE Trans. Sustain. Comput.*, vol. 8, no. 4, pp. 652-669, 2023.

[2] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Springer, 2017, pp. 409-437.